

CS 300 Project One: Pseudocode and Runtime Analysis

Vector Pseudocode

STRUCT Course

 courseNumber

 courseTitle

 prerequisites

END STRUCT

Create vector called courseList

FUNCTION LoadCourses(fileName)

 Open input file

 IF file cannot be opened

 Print "Error opening file."

 Return

 ENDIF

 Create list called allCourseNumbers

 WHILE file has more lines

 Read one line from file

 Split line into tokens using comma delimiter

 IF number of tokens is less than 2

 Print "Invalid file format."

 Continue to next line

 ENDIF

 Create Course object

 course.courseNumber = first token

```

    course.courseTitle = second token

    Add course.courseNumber to allCourseNumbers

    FOR each remaining token
        Add token to course.prerequisites
    ENDFOR

    Add course to courseList
ENDWHILE

END FUNCTION

FUNCTION PrintCourseList()

    Sort courseList alphabetically by course number

    FOR each course in courseList
        Print course.courseNumber + ", " + course.courseTitle
    ENDFOR

END FUNCTION

FUNCTION PrintCourseInformation(courseNumber)

    FOR each course in courseList
        IF course.courseNumber equals courseNumber
            Print course.courseNumber + ", " + course.courseTitle

            Print prerequisites

            Return
        ENDIF
    ENDFOR

END FUNCTION

```

Hash Table Pseudocode

Create HashTable called courseTable

FUNCTION LoadCourses(fileName)

 Read file line by line

 Validate format

 Create Course object

 Calculate hash key

 Insert course into hash table

END FUNCTION

FUNCTION PrintCourseList()

 Extract all courses from hash table

 Sort courses alphabetically

 Print sorted list

END FUNCTION

FUNCTION PrintCourseInformation(courseNumber)

 Calculate hash key

 Search hash table

 Print course information and prerequisites

END FUNCTION

Binary Search Tree Pseudocode

Create BinarySearchTree called courseTree

FUNCTION LoadCourses(fileName)

Read file line by line

Validate format

Create Course object

Insert course into binary search tree

END FUNCTION

FUNCTION PrintCourseList()

Perform InOrder Traversal

Print courses as visited

END FUNCTION

FUNCTION PrintCourseInformation(courseNumber)

Start at root node

Traverse tree

Print course information and prerequisites when found

END FUNCTION

Menu Pseudocode

BEGIN PROGRAM

Display Menu

1. Load Course Data File
2. Print Course List
3. Print Course Information
4. Exit

END PROGRAM

Runtime Analysis

Data Structure	Load File and Create Objects	Search for Course	Print Sorted Course List
Vector	$O(n)$	$O(n)$	$O(n \log n)$
Hash Table	$O(n)$	$O(1)$ average	$O(n \log n)$
Binary Search Tree	$O(n \log n)$	$O(\log n)$	$O(n)$

Advantages and Disadvantages

Vector

Advantages:

- Easy to implement and maintain
- Uses memory efficiently
- Sequential storage is straightforward

Disadvantages:

- Searching requires scanning the vector
- Sorting is required before printing courses alphabetically

Hash Table

Advantages:

- Fast average search performance
- Efficient insertion and retrieval
- Well suited for large datasets

Disadvantages:

- Data is not stored in sorted order
- Additional sorting is required before printing the course list

Binary Search Tree

Advantages:

- Maintains ordered data structure
- In-order traversal automatically produces sorted output
- Efficient searching and insertion

Disadvantages:

- More complex implementation
- Performance can degrade if the tree becomes unbalanced

Recommendation

The Binary Search Tree is the recommended data structure for the ABCU advising program.

While the hash table offers the fastest average search performance, the project requirements include displaying all courses in alphanumeric order. A binary search tree naturally maintains ordered data and allows an in-order traversal to produce a sorted course list without requiring additional sorting. This makes the binary search tree the best balance between efficiency and functionality for the advising system.